

EX. NO. 7**ENDORSEMENT POLICY CONFIGURATION IN HYPERLEDGER FABRIC****AIM**

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE**Part 1: OR Endorsement****Policy Step 1: Start Fresh**

Navigate to the Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-
```

network Stop any previously running

network.

```
./network.sh down
```

Start the Fabric network and create the default channel.

./network.sh up createChannel -ca

```
vboxuser@Ubuntu:~$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
```

Step 2: Set Environment Variables

Load the Org1 environment.

```
source setOrg1.sh
```

Set the TLS certificate for Org1.

```
export
```

```
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

Set the TLS certificate for Org2.

```
export
```

```
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

Step 3: Deploy Chaincode with OR Endorsement Policy

Deploy the Asset Transfer chaincode with a policy allowing either Org1 or Org2 to endorse transactions.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

```
channel 'mychannel' joined
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep "OR('Org1MSP.peer','Org2MSP.peer')"
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC VERSION: 1.0
```

Step 4: Verify Chaincode Commitment

Query the committed chaincode definition.

```
peer lifecycle chaincode querycommitted -C mychannel --name basic
```

Step 5: Initialize the Ledger

Since the policy is OR, endorsement from Org1 alone is sufficient.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"InitLedger","Args":[]}'
```

Step 6: Query All Assets

Retrieve all assets from the ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode
querycommitted -C mychannel --name basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Ap
provals: [Org1MSP: true, Org2MSP: true]
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafil
e $ORDERER_CA -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'
2026-08-31 16:11:11.650 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"d
ocType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad
","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3
","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"
yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValu
e":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":
15,"docType":"asset"}]
```

Step 7: Stop the Network

```
./network.sh down
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 10/10
✓ Container orderer.example.com      Removed      1.0s
✓ Container peer0.org1.example.com    Removed      1.9s
✓ Container ca_org1                  Removed      11.0s
✓ Container ca_org2                  Removed      10.8s
✓ Container ca_orderer               Removed      11.2s
✓ Container peer0.org2.example.com    Removed      1.6s
✓ Network fabric_test                Removed      0.3s
✓ Volume compose_peer0.org2.example.com Removed      0.0s
✓ Volume compose_orderer.example.com  Removed      0.0s
✓ Volume compose_peer0.org1.example.com Removed      0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing associated chaincode docker images
```

Part 2: AND Endorsement Policy

Step 8: Restart the Network

Start the Fabric network again.

`./network.sh up createChannel -ca`

```
deleted: sha256:89c3c4b4e3d0d1c72e15f4edeb0afab9d12f3f3d0d0d135c53d04b0de0f0c2277
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up create
Channel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay
of '3' seconds and using database 'leveldb with crypto from 'Certificate Author
ities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] up 4/4
✓ Network fabric_test    Created      0.1s
✓ Container ca_org2      Started     1.8s
✓ Container ca_org1      Started     1.5s
✓ Container ca_orderer   Started     1.3s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca
-org1 --tls.certfiles /home/vboxuser/fabric-dev/fabric-samples/test-network/orga
nizations/fabric-ca/org1/ca-cert.pem
2026/08/31 16:14:53 [INFO] Configuration file location: /home/vboxuser/fabric-de
```

Load the Org1 environment.

`source setOrg1.sh`

Set the TLS certificates.

export

ORG1_TLS=\${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export

ORG2_TLS=\${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt

```
channel mychannel joined
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ source setOrg1.sh
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
```

Step 9: Deploy Chaincode with AND Endorsement Policy

Deploy the chaincode with a policy requiring both organizations to endorse transactions.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

```
ts/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following

- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: AND('Org1MSP.peer','Org2MSP.peer')
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false

executing with the following

- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript

Step 10: Initialize the Ledger

Since the AND policy requires both organizations, the transaction must be sent to both Org1 and Org2 peers.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses
```

```
localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"InitLedger","Args":[]}'
```

Step 11: Create Asset25

Create the asset using endorsements from both organizations.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```

Wait for the transaction to complete successfully before querying the asset.



```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafil
e $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '
{"function":"InitLedger","Args":[]}'
2026-08-31 16:17:25.301 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafil
e $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '
{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
2026-08-31 16:17:31.821 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200 payload:{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}
```

Step 12: Verify the Asset

Query asset25 from the ledger.

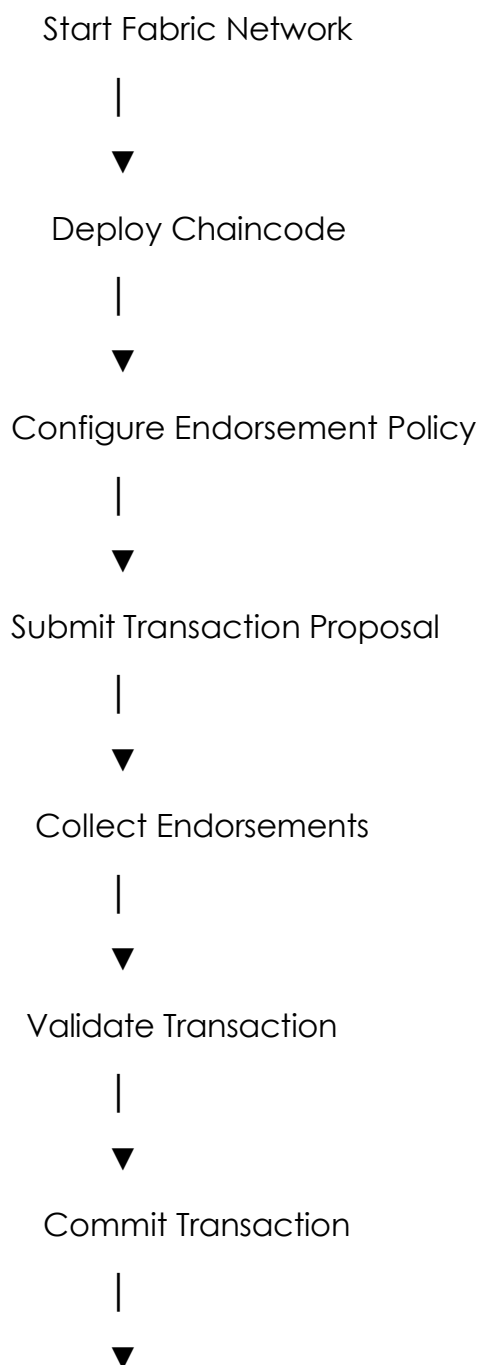
```
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset25"]}'
```

Step 13: Stop the Network

```
./network.sh down
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset25"]}'
{"AppraisedValue":1200,"Color":"Blue","ID":"asset25","Owner":"Arun","Size":10}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 3/6
✓ Container peer0.org1.example.com Removed 1.7s
✓ Container orderer.example.com Removed 1.0s
✗ Container ca_orderer Stopping 2.0s
✗ Container ca_org2 Stopping 2.0s
✗ Container ca_org1 Stopping 2.0s
✓ Container peer0.org2.example.com Removed 1.8s
```

FLOW DIAGRAM



Update Ledger

|



Query Ledger

|



Stop Network

RESULT

The endorsement policies were successfully configured and tested in the Hyperledger Fabric network. The OR policy demonstrated that either organization can endorse a transaction, while the AND policy demonstrated that endorsements from both organizations are required. The transactions were successfully validated and committed according to the configured policies, demonstrating secure and trusted transaction processing in a permissioned blockchain network.